Clone Python de bzip2, lzma et 7-zip

Analyse comparative de l'efficacité de la compression sur quatre types de fichiers

Cours	Codage et Traitement du Signal
Niveau	Licence 2 — Informatique, ESTI (Antananarivo)
Auteur	Jonathan
Date	24 octobre 2026
Livrables	compressor.py (CLI) — gui.py (interface animée) — ce rapport

Résumé. Ce rapport présente un utilitaire Python en ligne de commande qui reproduit le comportement de bzip2, lzma et 7-zip, ainsi qu'une interface graphique animée. Les trois algorithmes sont comparés sur un fichier texte, audio, image et vidéo selon la taille obtenue, le taux de compression et le temps d'exécution. Les formats déjà compressés (mp3, png, mp4) se recompressent très mal, alors que le texte brut est réduit d'environ 80 %.

1. Introduction

La compression **sans perte** représente une information avec moins de bits, de sorte que la décompression restitue exactement le fichier d'origine. Elle exploite la **redondance** des données : répétitions de motifs et fréquences inégales des symboles. Le projet demandé consiste à développer un clone fonctionnel de trois utilitaires de référence : **bzip2**, **lzma** et **7-zip**.

Taux de compression (%) = (1 − taille compressée / taille initiale) × 100. Un taux **négatif** signifie que le fichier a grossi.

2. Principe des trois algorithmes

Outil	Algorithme principal	Idée
bzip2	Transformée de Burrows-Wheeler (BWT) + Move-To-Front + RLE + Huffman	La BWT regroupe les symboles identiques ; MTF, RLE et Huffman exploitent ensuite cette régularité. Travail par blocs (jusqu'à 900 Ko).
lzma	LZ77 à grand dictionnaire + codage par intervalles (range coding)	Remplace les répétitions par des références (distance, longueur) vers des données déjà vues, puis code le tout avec un modèle probabiliste adaptatif.
7-zip (.7z)	LZMA2 (par défaut)	Évolution de LZMA : découpage en blocs, meilleure gestion des données incompressibles, multithreading possible.

3. Implémentation

3.1 Choix techniques

• Modules standards **bz2** et **lzma** de Python ; le vrai format **.7z** est produit avec la bibliothèque **py7zr** (à défaut, repli sur LZMA2 dans un conteneur .xz).
• Lecture par blocs de 1 Mo pour traiter les gros fichiers (vidéos) sans saturer la mémoire.
• Mesure du temps avec *time.perf_counter()* ; vérification d'intégrité par empreinte **SHA-256** après décompression.

3.2 Interface en ligne de commande

```
python compressor.py compress   <bzip2|lzma|7z> <fichier> [-o sortie]
python compressor.py decompress <bzip2|lzma|7z> <fichier> [-o sortie]
python compressor.py bench <fichier1> <fichier2> ... [--csv resultats.csv]
```

3.3 Structure du code

compress() et *decompress()* choisissent l'algorithme et traitent le fichier par blocs ; *report_line()* affiche tailles, taux et temps ; *cmd_bench()* exécute les trois algorithmes sur chaque fichier, vérifie l'intégrité et produit un tableau (et un CSV).

compressor.py — fonction compress()

```
$ sed -n '50,71p' compressor.py
def compress(algo, src, dst=None):
    dst = dst or src + EXT[algo]
    if algo == "bzip2":
        # BWT + MTF + Huffman, taille de bloc maximale (niveau 9 = 900 Ko)
        _stream(bz2.BZ2Compressor(9), src, dst)
    elif algo == "lzma":
        # LZMA1 : format historique ".lzma" (dictionnaire + codage par intervalles)
        _stream(lzma.LZMACompressor(format=lzma.FORMAT_ALONE, preset=9), src, dst)
    elif algo == "7z":
        if HAS_PY7ZR:
            with py7zr.SevenZipFile(dst, "w") as z:   # LZMA2 par défaut
                z.write(src, arcname=os.path.basename(src))
        else:
            _stream(lzma.LZMACompressor(format=lzma.FORMAT_XZ, preset=9), src, dst)  # LZMA2
    else:
        raise ValueError(f"Algorithme inconnu : {algo}")
    return dst


def decompress(algo, src, dst=None):
    if algo == "7z" and HAS_PY7ZR and src.endswith(".7z"):
        outdir = dst or os.path.dirname(os.path.abspath(src))
```

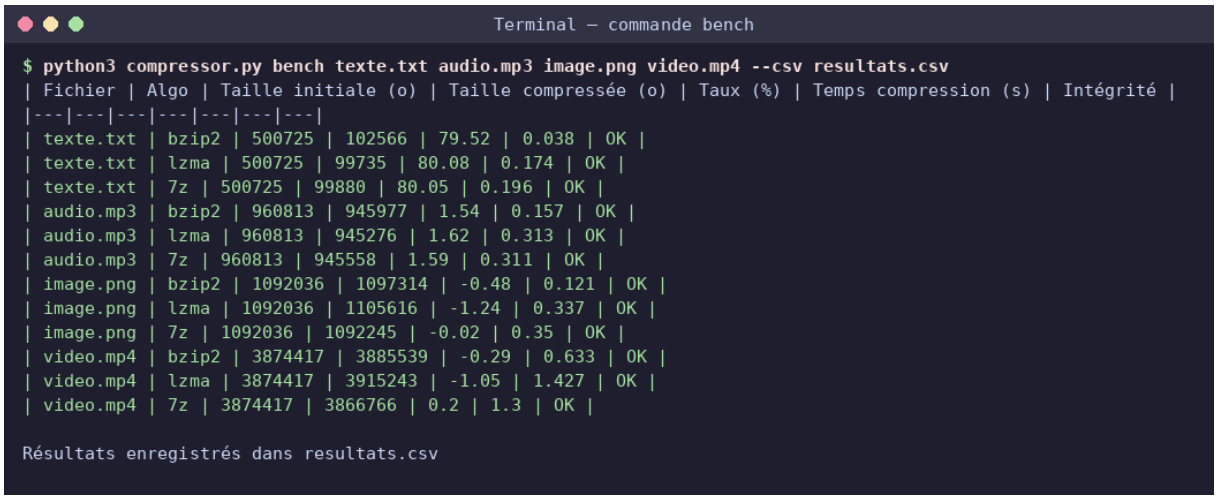
Capture 1 — Code source : fonction compress() de compressor.py (section 3.3)

4. Protocole expérimental

Machine : processeur Intel Xeon 2,8 GHz (1 cœur), Linux, Python 3.12.3, py7zr 1.1.4. **Niveau de compression :** maximal (9) pour les trois algorithmes. Chaque temps est la **médiane de 3 exécutions** ; les tailles sont exactes (en octets).

Type	Fichier	Taille	Description
Texte (.txt)	texte.txt	500 725 o	Documentation en anglais de Python (prose technique), encodage UTF-8
Audio (.mp3)	audio.mp3	960 813 o	60 s de signal synthétique (bruit rose + 2 sinusoïdes), encodé en MP3 128 kbit/s (LAME)
Image (.png)	image.png	1 092 036 o	Image 1280×720 générée (dégradés, bruit lissé, cercles), PNG optimisé
Vidéo (.mp4)	video.mp4	3 874 417 o	10 s de vidéo 1280×720 à 30 i/s (H.264, CRF 23) + audio AAC

Remarque : les fichiers audio, image et vidéo sont synthétiques pour garantir la reproductibilité. Ils sont encodés avec les vrais codecs (LAME, PNG/DEFLATE, x264/AAC) ; leur comportement face à la recompression est donc représentatif des fichiers réels, mais les valeurs exactes varieront avec d'autres fichiers.



Capture 2 — Commande bench : tableau complet avec colonne d'intégrité = OK (section 4). Chaque ligne affiche une exécution unique ; les tableaux de la section 5 donnent la médiane de 3 exécutions.

5. Résultats

5.1 Texte (.txt)

Algorithme	Taille initiale (o)	Taille compressée (o)	Taux (%)	Temps (s)
bzip2	500 725	102 566	79,52	0,040
lzma	500 725	99 735	80,08	0,217
7-zip (.7z)	500 725	99 880	80,05	0,203

5.2 Audio (.mp3)

Algorithme	Taille initiale (o)	Taille compressée (o)	Taux (%)	Temps (s)
bzip2	960 813	945 977	1,54	0,104
lzma	960 813	945 276	1,62	0,338
7-zip (.7z)	960 813	945 558	1,59	0,340

5.3 Image (.png)

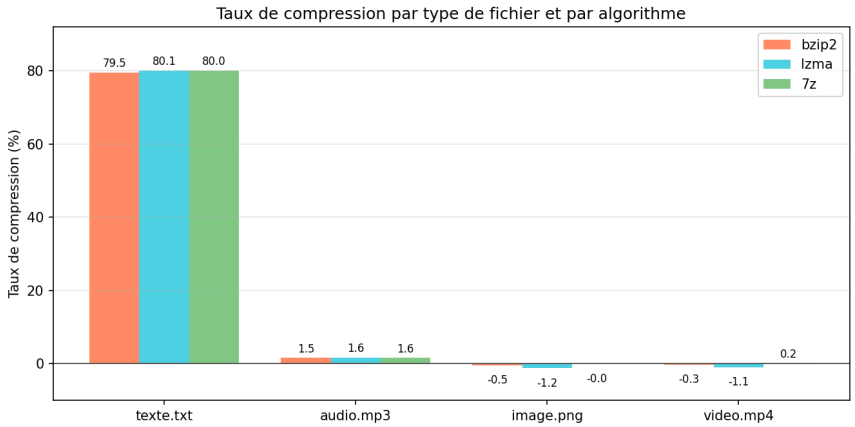
Algorithme	Taille initiale (o)	Taille compressée (o)	Taux (%)	Temps (s)
bzip2	1 092 036	1 097 314	-0,48	0,118
lzma	1 092 036	1 105 616	-1,24	0,370
7-zip (.7z)	1 092 036	1 092 245	-0,02	0,351

5.4 Vidéo (.mp4)

Algorithme	Taille initiale (o)	Taille compressée (o)	Taux (%)	Temps (s)
bzip2	3 874 417	3 885 539	-0,29	0,428
lzma	3 874 417	3 915 243	-1,05	1,219
7-zip (.7z)	3 874 417	3 866 766	0,20	1,276

5.5 Tableau de synthèse (taux de compression, %)

Type	bzip2	lzma	7-zip
Texte (.txt)	79,52	80,08	80,05
Audio (.mp3)	1,54	1,62	1,59
Image (.png)	-0,48	-1,24	-0,02
Vidéo (.mp4)	-0,29	-1,05	0,20



Capture 5 — Graphique des taux de compression par type de fichier (section 5.5)

5.6 Vérifications pratiques

```
Terminal — compression de texte.txt

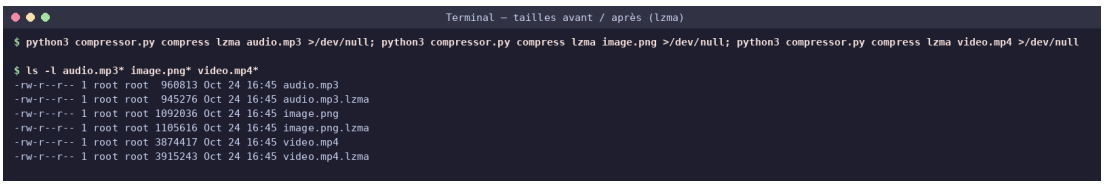
$ python3 compressor.py compress bzip2 texte.txt
[bzip2] texte.txt -> texte.txt.bz2
  Taille avant : 488.99 Ko (500725 octets)
  Taille après : 100.16 Ko (102566 octets)
  Taux de compression : 79.52 %   |   Temps : 0.048 s
$ python3 compressor.py compress lzma texte.txt
[lzma] texte.txt -> texte.txt.lzma
  Taille avant : 488.99 Ko (500725 octets)
  Taille après : 97.40 Ko (99735 octets)
  Taux de compression : 80.08 %   |   Temps : 0.182 s
$ python3 compressor.py compress 7z texte.txt
[7z] texte.txt -> texte.txt.7z
  Taille avant : 488.99 Ko (500725 octets)
  Taille après : 97.54 Ko (99880 octets)
  Taux de compression : 80.05 %   |   Temps : 0.218 s
```

Capture 3 — Compression de texte.txt avec chacun des trois algorithmes (section 5.1)

```
Terminal — décompression et vérification SHA-256

$ mkdir -p out && python3 compressor.py decompress bzip2 texte.txt.bz2 -o out/texte_bzip2.txt
[bzip2] texte.txt.bz2 -> out/texte_bzip2.txt   (temps : 0.016 s)
$ python3 compressor.py decompress lzma texte.txt.lzma -o out/texte_lzma.txt
[lzma] texte.txt.lzma -> out/texte_lzma.txt   (temps : 0.007 s)
$ python3 compressor.py decompress 7z texte.txt.7z -o out
[7z] texte.txt.7z -> out   (temps : 0.017 s)
$ sha256sum texte.txt out/texte_bzip2.txt out/texte_lzma.txt out/texte.txt | cut -c1-24,65-
20a68348d3dc7cae28616474  texte.txt
20a68348d3dc7cae28616474  out/texte_bzip2.txt
20a68348d3dc7cae28616474  out/texte_lzma.txt
20a68348d3dc7cae28616474  out/texte.txt
```

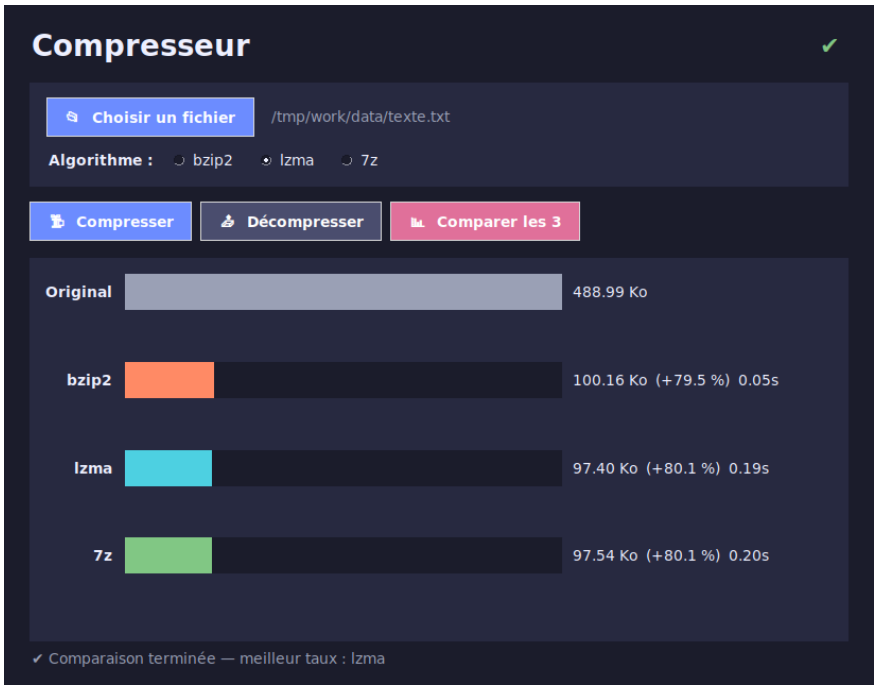
Capture 4 — Décompression puis vérification : les empreintes SHA-256 sont identiques, la compression est bien sans perte (section 5.1)



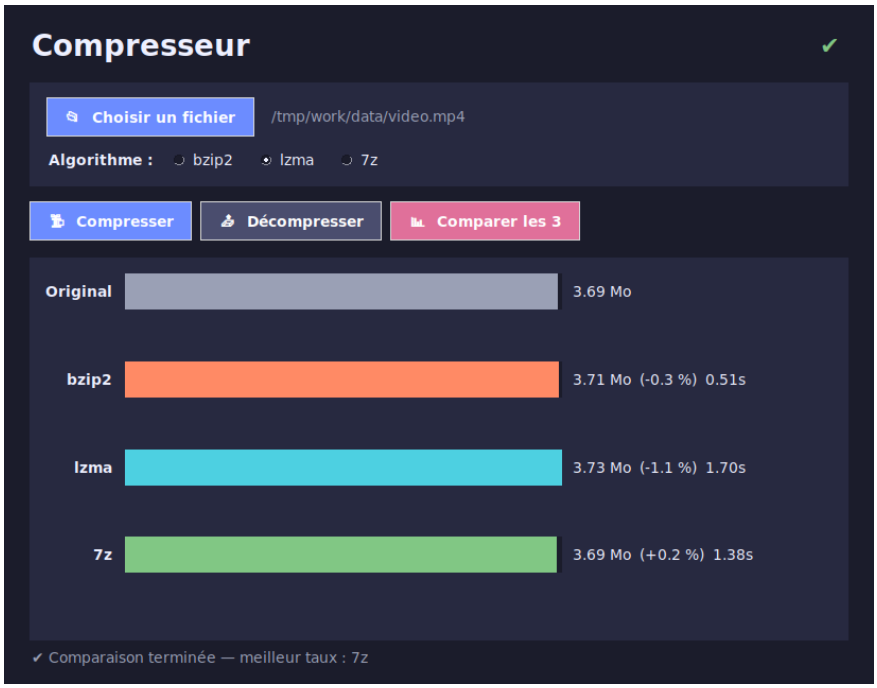
Capture 6 — Tailles des fichiers avant / après compression lzma (sections 5.2 à 5.4)

6. Interface graphique animée

En complément de la CLI, **gui.py** (Tkinter) propose une interface interactive : choix du fichier et de l'algorithme, boutons Compresser, Décompresser et « Comparer les 3 ». Les animations (spinner pendant le calcul, barres qui grandissent en cascade, compteur de pourcentage, fondu à l'ouverture) rendent la comparaison immédiatement lisible. Le calcul s'exécute dans un thread séparé, ce qui évite de figer la fenêtre sur les gros fichiers.



Capture 7 — Interface graphique, « Comparer les 3 » sur texte.txt : les barres compressées sont très courtes



Capture 8 — Même comparaison sur video.mp4 : les barres sont presque aussi longues que l'original

7. Analyse et discussion

7.1 Pourquoi le texte se compresse très bien

Le texte brut est très redondant : certaines lettres sont bien plus fréquentes que d'autres, et les mots et expressions se répètent. Les algorithmes à dictionnaire (LZ77/LZMA) et la BWT de bzip2 exploitent ces répétitions. Sur notre fichier texte, les trois algorithmes réduisent la taille d'environ 80 % (de 500 725 à environ 100 000 octets). lzma et 7-zip sont légèrement meilleurs que bzip2 (80,1 % contre 79,5 %), mais environ cinq fois plus lents à compresser (0,20 s contre 0,04 s).

7.2 Pourquoi .mp3, .png et .mp4 se recompressent mal

Ces formats sont **déjà compressés** : leur redondance a été éliminée par leur propre codec.

• **MP3** : compression *avec perte* (modèle psychoacoustique, quantification, codage de Huffman). Le flux obtenu ressemble à des données aléatoires. Gain observé : seulement 1,5 à 1,6 %.
• **PNG** : compression *sans perte* par filtrage puis **DEFLATE** (LZ77 + Huffman). Appliquer un second algorithme à dictionnaire n'apporte rien : les répétitions ont déjà été remplacées. Résultat : de −1,2 % à 0 % (le fichier grossit légèrement).
• **MP4** (H.264 + AAC) : compression *avec perte* très poussée (prédiction temporelle, transformée en cosinus, quantification, codage entropique). C'est le format le plus proche de l'aléatoire : résultat de −1,05 % à +0,2 %.

Un fichier dont les octets sont quasi aléatoires est **incompressible** : la limite d'entropie de Shannon interdit de descendre sous une certaine taille. Les taux négatifs s'expliquent par les en-têtes et structures que chaque format de compression ajoute. Le temps de calcul est en plus dépensé pour rien, surtout pour lzma sur la vidéo (≈ 1,2 s).

Observation : le mp3 gagne 1,5 % alors que png et mp4 ne gagnent rien. Notre mp3 est un signal synthétique avec peu de contenu varié (sinusoïdes), ce qui laisse probablement un peu de régularité dans le flux ; avec de la musique réelle, on attend un gain encore plus proche de 0.

7.3 Comparaison des algorithmes

• **bzip2** : bon compromis et rapide sur le texte ; ne détecte pas les longues répétitions éloignées.
• **lzma** : meilleur taux sur le texte grâce au grand dictionnaire, mais le plus lent et le seul à perdre plus de 1 % sur png et mp4.
• **7-zip (LZMA2)** : taux équivalent à lzma sur le texte, mais bien meilleur sur les données incompressibles (−0,02 % sur png, +0,2 % sur mp4), car LZMA2 stocke tels quels les blocs incompressibles.

7.4 Limites

• Un seul fichier par catégorie, avec des fichiers de test synthétiques (sauf le texte) : les résultats sont indicatifs, pas statistiques.
• Les temps dépendent de la machine et de la charge système (1 cœur, ici).
• Le clone s'appuie sur les bibliothèques bz2, lzma et py7zr ; il ne réimplémente pas la BWT ni le codage par intervalles.

8. Conclusion

La compression sans perte est très efficace sur les données redondantes (texte : environ 80 %) et pratiquement inutile sur les formats déjà compressés (mp3, png, mp4 : −1,2 % à +1,6 %). Le choix de l'algorithme dépend du compromis entre taux, temps et mémoire : bzip2 pour la rapidité avec un bon taux sur le texte, lzma pour la meilleure réduction, et 7-zip (LZMA2) pour un comportement plus sûr sur des données mixtes ou déjà compressées. En pratique, il est inutile de recompresser des fichiers mp3, png ou mp4.

Annexes

A. Code source : compressor.py (CLI) et gui.py (interface). **B.** Résultats bruts : resultats.csv. **C.** Références : documentation Python des modules bz2 et lzma, documentation py7zr, documentation de 7-zip.